Serving Qwen3.8-27B on a Single RTX 4090: 2.5× Faster Decoding and Twice the Context at Baseline Precision

The qwen-4090-bench project
https://github.com/kineticsystem/qwen-4090-bench

October 2026

Abstract

We study how fast, and with how much context, a dense 27B-parameter hybrid model can be served to a single user on one 24 GB RTX 4090 without losing precision. Starting from a tuned llama.cpp setup (89.6 tok/s, 128K context), we compare three specialized engines on Qwen3.8-27B: OpenJet, a persistent megakernel that reads the same GGUF file; NInfer, a dedicated engine with its own 4/5-bit weights; and azllm, a tqLLM fork that reads rotated 4.3-bit weights. Both NInfer and azllm draft with DFlash2, a block-diffusion model. azllm decodes at 228.6 tok/s on short prompts (2.55×), up to 3.1× on code, and 2.47× in thinking mode; NInfer reaches 1.85× and OpenJet 1.52×. On the same 98K positions of code, azllm's weights lose 0.46% in log-likelihood against the baseline's and NInfer's 1.78%, and GSM8K and HumanEval stay within the noise of greedy decoding. A capacity map over KV formats, drafters and the vision tower, verified by filling each context with real source code, shows that the model's full 262,144-token window fits with vision in two engines. A 4-bit KV cache costs 0.16% (azllm) to 0.82% (NInfer) in log-likelihood on code, an effect invisible to exact recall of code at 252K tokens. We release launcher scripts for every configuration.

1 Introduction

A dense 27-billion-parameter model quantized to about 4.5 bits per weight occupies roughly 17 GB, which leaves about 6 GB of a 24 GB consumer GPU for the key–value (KV) cache, the activations and the engine's own buffers. At batch size one, decoding is bound by memory bandwidth: every generated token reads every weight once. On an RTX 4090 (1008 GB/s), this puts plain autoregressive decoding of such a model at about 50–60 tokens per second, whatever the engine.

Two levers move this limit without changing the weights. *Speculative decoding* lets a cheap drafter propose several tokens that the target model verifies in one forward pass, which costs about as much memory traffic as a single token. And *engine specialization* removes the overheads a general engine carries: kernel launches, host round trips and generic memory layouts. A third lever, *KV-cache quantization*, does not make short prompts faster, but decides how much context fits in the memory that is left.

This report studies the three levers on one concrete case: Qwen3.8-27B, a hybrid Gated-DeltaNet/attention model with a built-in multi-token-prediction (MTP) head, served for a single user who mostly writes code, on one RTX 4090. We start from a tuned llama.cpp setup and ask three questions:

1. How much faster can single-stream decoding get, at unchanged accuracy?
2. How much context fits on the card, with and without the vision encoder, without the server failing?
3. What does a 4-bit KV cache cost in precision on code, compared with the 8-bit keys and 4-bit values of the starting setup?

We compare four engines: llama.cpp with the TurboQuant KV types [2, 6] (the baseline), the OpenJet persistent megakernel [1], the RTX 4090 port of NInfer [4, 5] with both MTP and the DFlash2 block-diffusion drafter, and azllm [8], a fork of tqLLM [7] that reads rotated TurboQuant weights. All measurements come from one machine, in throw-away containers, with scripts that are part of this work.

Contributions.

- A like-for-like measurement of decode speed, accuracy and context capacity for four engines serving the same model on the same GPU, including thinking mode and a 29K-token prompt.
- A comparison of three 4-bit weight files on exactly the same code positions, showing that a rotated 4.3-bit format predicts code within 0.5% of a 5.2-bit k-quant file.
- A capacity map: the largest context each configuration (engine, KV type, drafter, vision on or off) accepts, each verified by filling the context with real code and querying it.

Table 1: KV-cache formats compared. Bits are per element, scales excluded.

Engine	Format	K bits	V bits	Encoding
llama.cpp	`q8_0/q8_0`	8	8	blocks of 32, one scale
llama.cpp	`q8_0/turbo4`	8	4	TurboQuant values (baseline)
OpenJet	`q8`	8	8	as `q8_0`
NInfer	`int8`	8	8	groups of 64
NInfer	`rk8v4`	8	4	rotated keys, groups of 64
NInfer	`rk4v4-e8`	4	4	rotated, E_8 lattice keys

- A measurement of the cost of KV-cache quantization on code perplexity at 64K and 128K context, and on exact recall of code at up to 252K tokens.

- Ready-to-run server scripts for every configuration; the recommended one serves 2.5× faster than the baseline with 1.9× its context and vision, and the full 262K window with a 4-bit KV cache.

2 Background

The model. Qwen3.8-27B interleaves Gated DeltaNet layers, whose state is a fixed-size matrix per head, with full-attention layers, whose KV cache grows with the context. Only the attention layers pay per-token memory, which is why a 27B model can hold a 256K-token context in a few gigabytes. The recurrent state, however, cannot simply be truncated when a draft token is rejected: engines must either snapshot it or replay the accepted tokens.

Speculative decoding. A drafter proposes k tokens; the target model scores all of them in one pass and keeps the longest prefix that matches its own choice (greedy) or passes a rejection test (sampling), so the output distribution is unchanged. The speed-up depends on the mean number of tokens accepted per pass and on how cheap the drafter and the $(k{+}1)$-token verification are. Qwen3.8-27B ships an MTP head that drafts one token per step from the target's last hidden state. DFlash2 is a separate small block-diffusion model that drafts a whole block in one pass, conditioned on the target's hidden states, and is accepted further on predictable text such as code.

KV-cache quantization. The KV cache can be stored in fewer bits than the activations that produce it. Keys are more sensitive than values, since an error in a key changes every attention weight it takes part in. Rotations spread outliers across channels before quantization, and lattice codes such as E_8 quantize eight values jointly at a lower error than scalar rounding at the same rate. Table 1 lists the formats compared in this work.

3 Systems Under Test

Figure 1 shows the four engines side by side. All four serve an OpenAI-compatible HTTP API, run one request at a time, and verify drafts exactly.

Baseline: llama.cpp with TurboQuant. The starting point is the TurboQuant fork of llama.cpp [6], built from commit `a3d5603d1` for `sm_89`, serving Unsloth's `UD-Q4_K_XL` GGUF with flash attention, MTP speculation with two drafts, keys in `q8_0`, values in `turbo4` and a 128,000-token context. The vision projector stays in host memory.

OpenJet Engine. OpenJet [1] runs a whole speculative cycle (verification of $1 + k$ tokens, lm-head, acceptance, MTP catch-up and the next k drafts) inside one cooperative kernel separated by software grid barriers. It reads the same GGUF, keeps the k-quant encodings, and rolls the DeltaNet state back by replaying accepted drafts from a small ring instead of snapshotting it. The draft lm-head is restricted to the 32,768 most frequent tokens; verification uses the full vocabulary. It is text only, and its KV cache is `f16` or `q8`.

NInfer-4090. NInfer [4] is a C++20/CUDA engine written for the Qwen3.6/3.8 family; we use the `sm_89` port [5]. It loads its own groupwise artifact (4- and 5-bit weight groups, 8-bit output head) and offers MTP or DFlash2 drafting, a paged KV cache in several formats (Table 1), prefix reuse at message boundaries, and an optional vision tower. Its image targets CUDA 13.1; on the 580 driver (CUDA 13.0) it runs through minor-version compatibility once the container's requirement check is disabled.

azllm (tqLLM). tqLLM [7] is a C++/CUDA engine for Qwen models stored in its own rotated TurboQuant weight types, a GGUF extension that stock llama.cpp cannot load. azllm [8] is a fork that changes only the serving layer (streaming with tools, live timings, prompt-cache bookmarks); we found it through a practitioner's report. We serve the `TQ_L` weights (4.31 bits per weight, 14.7 GB, against about 5.2 for the other two files) with a DFlash2 drafter quantized to TQ3 (0.79 GB) and a 65,536-token draft vocabulary. KV formats pair rotated 8- or 4-bit keys with 8-, 4-, 3- or 2-bit values; the prompt cache keeps the previous request's KV so that a repeated prefix is not read again. The vision projector must be F16 or F32. `TQ_L` embeds Qwen's official chat template, which fails on tool calls whose arguments arrive as a JSON string, so our launcher passes the fixed template that the `TQ3_4S-v2` file embeds.

Engines not retained. A Windows-oriented fork of NInfer-4090 has the same single-token decode rate (about

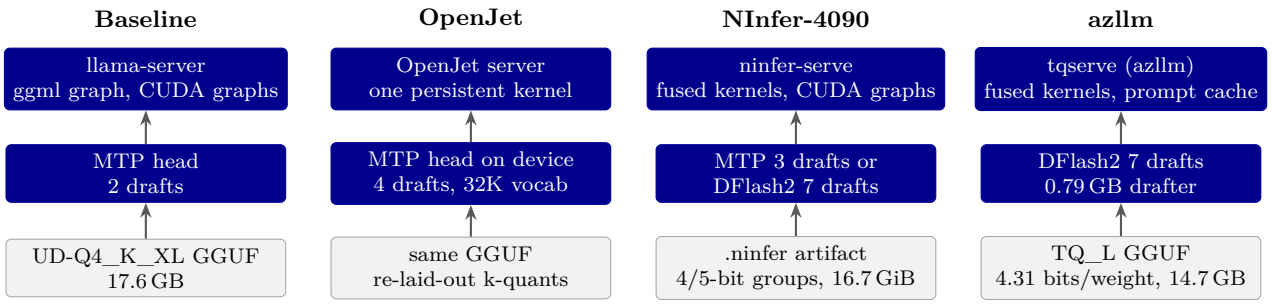


Figure 1: The four engines. Grey: the weights each one reads; blue: the drafter and the serving layer. OpenJet reads the baseline's own GGUF file; NInfer and azllm need their own weight files, both of which come with a DFlash2 drafter.

52 tok/s) and its KV modes are already merged into the port we use. vLLM and SGLang with 4-bit AWQ/GPTQ weights leave too little memory for a useful KV cache on 24 GB, and NVFP4 needs Blackwell. Lucebox runs DFlash2 inside a llama.cpp-based server but publishes no RTX 4090 figure, and the one community report of DFlash2 in llama.cpp on this card we found gives 73–75 tok/s.

4 Method

Hardware and isolation. One NVIDIA GeForce RTX 4090 (24 GB, 450 W stock limit, PCIe 3.0 x16), an Intel Core i9-10900K and 62 GB of RAM, under Ubuntu with driver 580.178. Every engine is built and run in a throw-away Docker container; the model files are mounted read-only. One server runs at a time.

Decode speed. A client streams each answer and computes decode speed as $(n-1)/(t_{\text{last}} - t_{\text{first}})$, where n is the number of generated tokens and t the arrival times of the first and last streamed tokens, so prompt processing is excluded and every engine is measured the same way. The short benchmark has six prompts (two coding tasks, a short story, a technical explanation, a word problem and a table), up to 1024 tokens each, with the sampling the model card recommends (temperature 0.6, top-p 0.95, top-k 20, min-p 0, presence penalty 0) and a fixed seed. We report the mean over the prompts. Final configurations were repeated three times; with a fixed seed the repetitions agree within 1 tok/s, while separate server starts vary by about 5%. The long-prompt benchmark prefixes three questions with 29,005 tokens of CUDA source code.

Accuracy. GSM8K (first 250 test problems) and HumanEval (all 164, pass@1, run against the official tests), greedy, thinking off.

Context capacity. For NInfer, the memory planner decides at startup whether a configuration fits, so we binary-search the largest `-max-context` (in steps of 4096 tokens) that starts. For OpenJet, which allocates at startup and fails with `cudaMalloc` out of memory, we do the same. A start is not proof: each retained configuration is then filled to within 2–3% of its limit with real source code and queried (next paragraph); we call a context *usable* only if the server answers every question and is still alive afterwards.

KV precision. Two measurements. (i) Code perplexity: NInfer's offline evaluator scores 359,927 tokens of CUDA/C++ source with windows of 64K (stride 32K) or 128K (stride 64K) tokens, for each KV format; we patched its command line to accept the rotated and lattice formats that the server already supports. For llama.cpp, `llama-perplexity` scores the same text in 64K chunks. For azllm, `tqkld` first dumps the top-16 log-probabilities of the `q8` cache as a reference, then scores the other formats against it (KL divergence, top-1 agreement and NLL), on four 64K chunks of which the last 32K positions are scored. Within each engine, the most precise format it offers is the reference. (ii) Long-context recall: the prompt holds up to 252K tokens of llama.cpp source, then twelve questions ask the model to copy, exactly, the first 16 lines of the body of a named function in a named file, at depths spread from 10% to 98%. A function qualifies only if its name is defined once in its file. We report the share of lines copied exactly.

Weight precision. The three weight files are compared on the same 98,304 positions: chunks 1–3 of 64K tokens, last 32K of each, with 8-bit keys and values. `llama-perplexity` with 64K chunks, `tqkld -tail 32768`, and NInfer's windows starting at 64K, 128K and 192K (stride 32K) all score exactly these positions, and the three tokenizers produce the same 359,928 tokens. The comparison includes each engine's own arithmetic, not the weights alone, and there is no BF16 reference: it tells which pipeline predicts the code best, not which is closest to the original model.

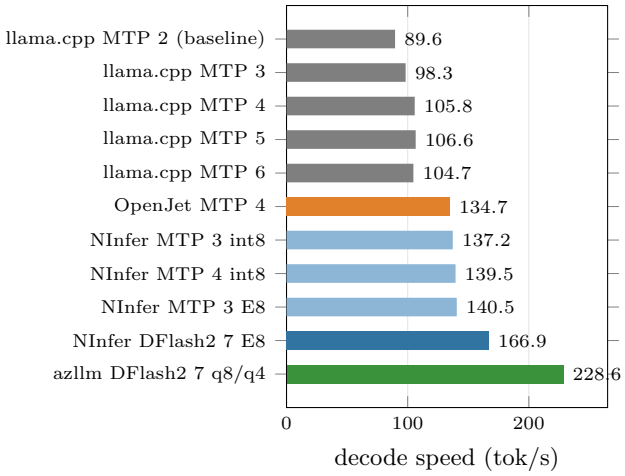


Figure 2: Mean decode speed over six short prompts, recommended sampling, thinking off. Grey: llama.cpp; orange: OpenJet; blue: NInfer; green: azllm.

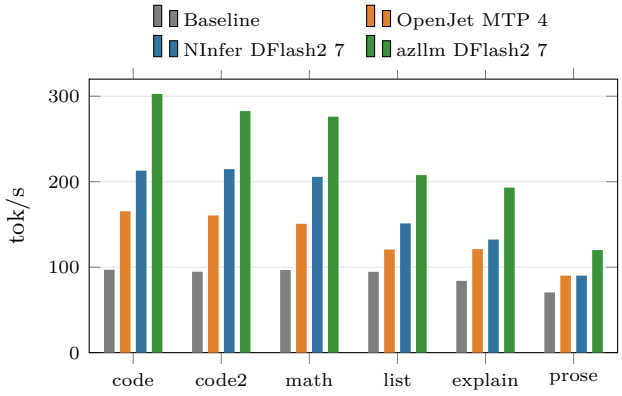


Figure 3: Decode speed per prompt. Structured output (code, a worked problem, a table) gains most; a short story gains least.

5 Results

5.1 Decode speed at short context

Figure 2 ranks the configurations on the six short prompts. Raising the baseline's MTP drafts from two to five gains 19%; beyond five, rejected drafts cost more than they bring. OpenJet, with four MTP drafts on the baseline's own GGUF, reaches 134.7 tok/s (1.50×). NInfer with MTP is slightly faster (137–140 tok/s) and with seven DFlash2 drafts reaches 166.9 tok/s (1.86×). azllm, with seven DFlash2 drafts on its smaller `TQ_L` weights, reaches 228.6 tok/s: 2.55× the baseline and 1.37× NInfer.

The gain depends on the text (Figure 3). On the two coding prompts and the word problem, azllm runs 2.9–3.1× faster than the baseline and NInfer 2.1–2.3×; on free prose, which no drafter predicts well, the gains fall to 1.70× and 1.28×. Thinking mode (effort *medium*) gives 208.4 against 84.2 tok/s for azllm (2.47×) and 153.3 for NInfer (1.82×).

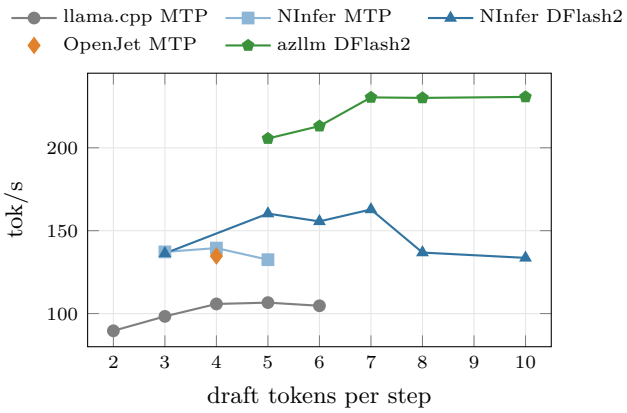


Figure 4: Decode speed against the number of draft tokens. NInfer DFlash2 points use E8 KV, except 3 and 10 drafts (int8 KV at 96K); azllm points use `q8/q4` KV at 128K without vision.

Number of drafts. Figure 4 sweeps the number of draft tokens. NInfer's DFlash2 peaks at five to seven drafts and drops sharply beyond; azllm's DFlash2 rises up to seven and stays flat up to ten. MTP, which drafts one token per step, peaks at three to five. On NInfer the KV format makes no measurable difference at short context (DFlash2 7: 166.9 tok/s with E8, 166.7 with int8); on azllm neither (228.5 with `q8/q4`, 230.7 with `q4/q4`).

5.2 Accuracy

Table 2 lists accuracy next to speed for the retained configurations. With greedy decoding and thinking off, all engines solve 238–244 of the 250 GSM8K problems and 156–158 of the 164 HumanEval problems. OpenJet, which reads exactly the baseline's weights, solves four GSM8K problems fewer than the baseline, and azllm solves more with its 4-bit cache (242) than with its 8-bit one (238), so differences of this size are the noise of greedy decoding on near-ties. To compare the weights themselves we turn to perplexity.

5.3 Weight precision

Figure 5 compares the three weight files on the same 98,304 positions of code. Taking the baseline's UD-Q4_K_XL as the reference (0.4330 nats per token), azllm's `TQ_L` loses 0.46% and NInfer's groupwise artifact 1.78%. The ranking holds in every chunk. `TQ_L` is the smallest file (4.31 against about 5.2 bits per weight), so its rotated format buys both speed and precision. Prefilling with FP16 instead of int8 activations changes azllm's result by 0.0002 nats, so the gap comes from the weights, not the prefill path.

Table 2: The retained server configurations, as started by the launcher scripts. Speeds in tok/s (speed-up over the baseline in parentheses); accuracy greedy, thinking off.

Configuration	KV (K/V bits)	Context	Vision	Drafter	Short	Thinking	29K prompt	GSM8K	HumanEval
Baseline (llama.cpp)	q8_0/turbo4 (8/4)	128,000	yes	MTP 2	89.6	84.2	76.0	97.2	96.3
OpenJet	q8 (8/8)	131,072	no	MTP 4	135.9 (1.52×)	130.2	110.7	95.6	96.3
NInfer `precise`	rk8v4 (8/4)	204,800	yes	MTP 3	137.4 (1.53×)	129.7	122.2	97.6	95.1
NInfer `long`	E8 (4/4)	262,144	yes	MTP 3	139.0 (1.55×)	131.0	121.8	96.0	96.3
NInfer `fast`	E8 (4/4)	192,512	yes	DFlash2 7	166.0 (1.85×)	153.3	132.8	97.2	95.7
azllm	q8/q4 (8/4)	241,664	yes	DFlash2 7	**228.6** (2.55×)	**208.4**	**150.1**	95.2	95.7
azllm, full window	q4/q4 (4/4)	262,144	yes	DFlash2 7	228.8 (2.55×)	195.1	145.6	96.8	95.7

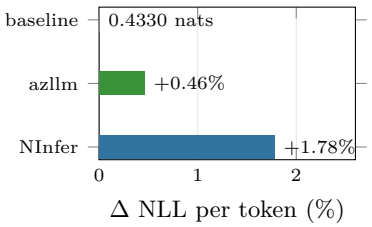


Figure 5: Mean NLL per token on 98,304 positions of C++/CUDA code (32K–64K tokens of context each), relative to the baseline's weights; 8-bit KV in every engine. Lower is better.

5.4 Context capacity

Figure 6 maps the largest context each configuration accepts. Three factors decide it: the KV format (int8 to rk8v4 to E8 roughly doubles NInfer's capacity), the drafter (NInfer's DFlash2 keeps its own KV cache and costs 57–111K tokens), and the vision tower (16–33K tokens). OpenJet allocates a fixed 2048-token prefill scratch; shrinking it to 512 tokens raises its ceiling from 131,072 to 147,456 tokens with `q8` keys and values, and it has no 4-bit KV and no vision. azllm reaches the full native window, with vision, with `q4/q4` KV, and 241,664 tokens with `q8/q4`. Near its limits it moves the token embeddings and a copy of the LM head to host memory; decode speed is unchanged (228.5 tok/s at 241,664 tokens with vision, 230.4 at 131,072 without).

Does the context really fit? A configuration that starts may still fail later, when a long prompt allocates prefill buffers. Table 3 fills each retained configuration to near its limit and queries it. Every server answered every question and stayed up, and the vision-enabled ones still described an image afterwards. The baseline slows to 55 tok/s at 121K tokens; NInfer stays above 106 tok/s at 252K, and azllm answers at 162 tok/s with 252K tokens in context. azllm's prompt cache also shows here: after reading 219K tokens in 147 s, the next request, the same prompt plus 15K tokens, was read in 15 s.

5.5 What a 4-bit KV cache costs on code

The recall test of Table 3 does not separate the KV formats: at 98K tokens, int8, rk8v4 and E8 all copy every requested line exactly, and the 4-bit formats of both NInfer and azllm still do at 252K. Perplexity is more sensitive (Figure 7). In NInfer, int8 is indistinguishable from bf16, rk8v4 costs 0.13% (0.15% at 128K context), and E8 with 4-bit keys 0.82% (0.79%). In llama.cpp, the baseline's `turbo4` values cost 0.11%. In azllm, `q8/q4` costs 0.04% and `q4/q4` 0.16%, with a mean KL divergence from the `q8` cache of 0.0008 and 0.0019 nats and top-1 agreement of 99.5% and 99.1%. azllm's 4-bit keys, rotated and quantized in blocks of 32 with one FP16 scale, are five times cheaper than NInfer's E_8-lattice keys.

Table 3: Full-context runs: prompt filled with source code, twelve questions asking for the first 16 lines of a named function. "Exact": answers with all 16 lines in place; "Similar": mean line-sequence similarity; decode speed while answering.

Configuration	Prompt	Exact	Similar	tok/s
Baseline, vision	121,375	11/12[a]	0.989	55.3
OpenJet q8, 147K	142,807	12/12	1.000	125.1
NInfer fast, vision	184,886	12/12	1.000	114.7
NInfer precise, vision	196,772	12/12	1.000	109.9
NInfer precise, text	221,095	11/12[b]	0.995	106.3
NInfer long, vision	252,392	12/12	1.000	108.4
azllm `q8/q4`, vision	233,777	11/12[a]	0.989	132.1
azllm `q4/q4`, vision	252,392	12/12	1.000	161.8

[a] The function contains the literal text `<|im_start|>`, which llama.cpp and azllm read as a special token and drop from the output; the rest was exact. [b] The answer omitted the first line, `try {`, and copied the other 15 exactly.

6 Discussion

Where the speed comes from. Single-token decode of a 17 GB model is bound by memory bandwidth at about 50 tok/s on this card, and every engine sits close to that bound without speculation (NInfer 50.5 tok/s, llama.cpp 45.9, as reported by the NInfer port). All of the gain therefore comes from three places: how many tokens a verification pass accepts (the drafter), how cheap the drafter and the $(k{+}1)$-token verification are (the engine), and how many bytes a pass reads (the weights). DFlash2 accepts longer runs than the model's own MTP head on structured text, and both specialized engines make verification cheap. azllm adds the third lever: its weight file is 16% smaller than the baseline's, and on code it

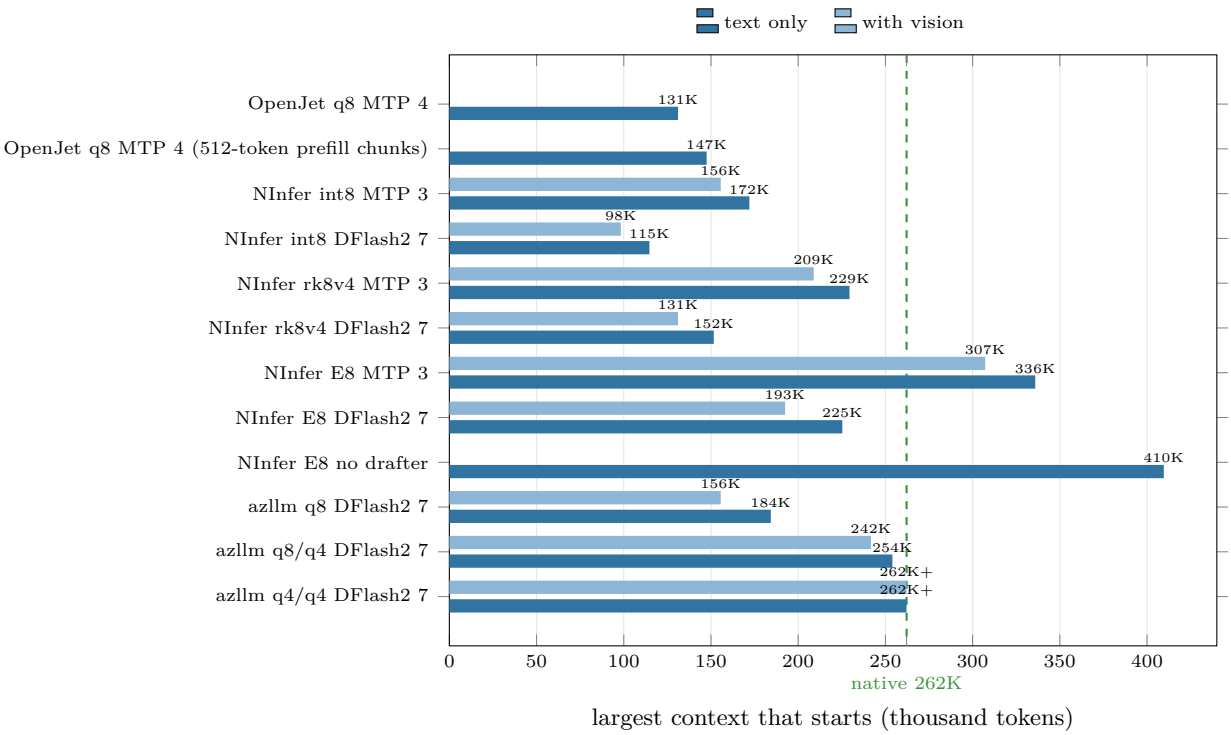


Figure 6: Largest context accepted at startup, binary-searched in steps of 4096 tokens. The baseline's own setting is 128,000 tokens. OpenJet has no vision; the no-drafter E8 row is shown text only; azllm's search stopped at the native window. The dashed line marks the model's native window; going past it would need RoPE scaling, which we did not test.

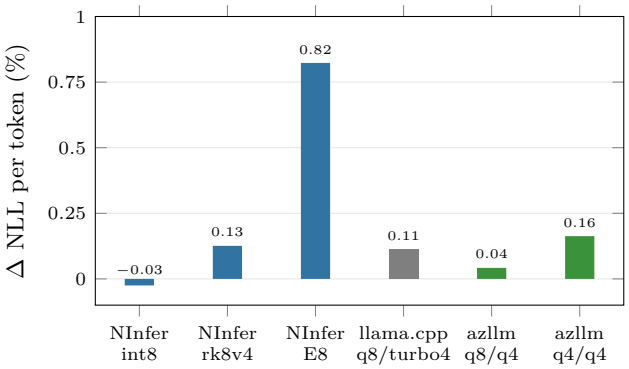


Figure 7: Cost of each KV format on code: relative change of the mean NLL per token against the most precise format of the same engine (NInfer: bf16; llama.cpp: q8_0/q8_0; azllm: q8), 64K windows with the last 32K positions scored. Lower is better.

predicts nearly as well, which is unusual for a smaller file and points to the value of rotation before quantization.

Context is a memory budget. On 24 GB, the weights take 15–18 GB and everything else competes for the rest: the KV cache, the drafter and its own cache, the vision tower, prefill buffers and CUDA graphs. Two choices dominate. 4-bit keys double the context, and a block-diffusion drafter with its own KV cache costs up to 111K tokens in NInfer. azllm's drafter is small (0.79 GB), so its fastest configuration also reaches the full window. Startup checks are not enough: we filled every retained configuration with real code before calling its context usable.

Precision of a 4-bit KV cache. For code, the KV cache matters at long range, where an error in a key shifts attention to the wrong line. Exact recall of 16-line functions did not separate any format, even at 252K tokens. Perplexity did: a 4-bit cache costs between 0.16% (azllm) and 0.82% (NInfer's E_8), against 0.11% for the baseline's 4-bit values. The difference between the two 4-bit formats is larger than the difference between 4- and 8-bit keys within azllm, so the format matters more than the bit count.

Limitations.

- One machine, one GPU, one session. Accuracy runs are single greedy passes; the GSM8K and HumanEval differences we report are of the size of their noise.
- There is no BF16 reference model: the weight comparison ranks three 4-bit pipelines, each with its own arithmetic, not their distance to the original model.
- We did not evaluate long agentic sessions with tools. Field reports describe looping with four or more

MTP drafts in NInfer during long tool-calling runs; our NInfer profiles use three, and azllm limits repetition by default (`-repeat-limit 256`). A tool-call smoke test passed on azllm with the fixed template.

- Speeds were measured with the card at its stock 450 W limit. The host reset once, without a GPU error in the log, during back-to-back compute-bound runs; thereafter a guard logged temperatures every 10 s (peak 80 °C) and never had to pause.
- azllm and NInfer are young projects maintained by few people, and both need weight files of their own.

7 Conclusion

On one RTX 4090, Qwen3.8-27B can be served 2.5× faster than a tuned llama.cpp setup, with almost twice its context and vision, at the same precision on code. The engine that achieves it, azllm, combines a DFlash2 block-diffusion drafter, a specialized CUDA engine and a rotated 4.3-bit weight format that predicts code within 0.5% of the baseline's 5.2-bit file. Its 4-bit KV cache costs 0.16% in log-likelihood and fits the full 262K-token window with the vision tower. We release launcher scripts for every configuration measured here, served behind llama-swap [3], which starts the engine a request asks for.

References

[1] L. Forster. OpenJet engine. `https://github.com/L-Forster/open-jet`, 2026. commit 72ba4a7.

[2] ggml-org. llama.cpp. `https://github.com/ggml-org/llama.cpp`, 2026. tag b10246.

[3] mostlygeek. llama-swap. `https://github.com/mostlygeek/llama-swap`, 2026. commit c31d416.

[4] Neroued. NInfer. `https://github.com/Neroued/ninfer`, 2026.

[5] sergiuszm. NInfer-4090. `https://github.com/sergiuszm/ninfer-4090`, 2026. commit fdb7a6cf.

[6] TheTom. llama-cpp-turboquant. `https://github.com/TheTom/llama-cpp-turboquant`, 2026. commit a3d5603d1.

[7] turbo tan. tqLLM. `https://github.com/turbo-tan/tqllm`, 2026.

[8] xhinker. azllm. `https://github.com/xhinker/azllm`, 2026. commit 1bce696.

A Reproducing the Measurements

All paths are relative to `~/repo/qwen-research`. Each engine runs in a throw-away container; one server at a time.

What	Command
Start a server (port 8084)	`./qwen-baseline.sh start`, `./qwen-openjet.sh start`, `./qwen-ninfer.sh start long|fast|precise`, `./qwen-azllm.sh start [-kv q4/q4 -ctx 262144]`
Short-prompt speed	`bench/bench.py -url http://127.0.0.1:8084 -model Qwen3.8-27B -label X [-think] [-greedy]`
29K-token prompt	`bench/bench.py... -long 100000`
GSM8K and HumanEval	`bench/evalacc.py -url... -label X`
Full-context recall	`bench/longctx.py -url... -tokens N -n-questions 12 -lines 16`
NInfer capacity	`bench/probe-ninfer.sh "label|-kv-dtype rk4v4-e8 -spec mtp -draft-tokens 3"`
OpenJet capacity	`DOCKER_ENV="-e MK_PF_CAP=512" bench/probe-openjet.sh "label|-kv q8 -d 4"`
Code perplexity, NInfer	`bench/ppl-ninfer.sh 65536 32768 int8 rk8v4 rk4v4-e8`
Code perplexity, llama.cpp	`bench/ppl-llama.sh 65536 q8_0/q8_0 q8_0/turbo4`
azllm capacity	`bench/probe-tqllm.sh "label|-kv q8/q4"`, then `bench/check-tqllm-fit.sh`
KV cost, azllm	`bench/kld-tqllm.sh q8 q8/q4 q4/q4`
Weight comparison	`bench/weights-compare.sh`, then `python3 bench/weights-compare.py`
Temperature guard	`nohup bench/thermal-guard.sh &`

For NInfer, `bench/bench.py`, `evalacc.py` and `longctx.py` take `-no-think top`: NInfer rejects `chat_template_kwargs` and reads `enable_thinking` and `reasoning_effort` as top-level request fields. Raw results are in `results/`: `bench_summary.jsonl` (speed), `accuracy.jsonl`, `longctx.jsonl` (with every answer), `ninfer-capacity.txt`, `openjet-capacity.txt`, `tqllm-capacity.txt`, `tqllm-fit.txt`, `ppl-ninfer.txt`, `ppl-llama.txt`, `tqkld.txt`, `weights-compare-table.txt` and `thermal.log`.

A note on stability. The host reset once without any GPU error in the kernel log, during back-to-back compute-bound perplexity runs at the card's stock 450 W limit. The scripts now pause 60 s between heavy runs and a guard pauses the containers above 83 °C on the GPU or 90 °C on the CPU; no further reset occurred and the guard never triggered.